

Die Aufgaben 1 bis 4 beziehen sich auf die folgende Ausgangssituation:

Ein landwirtschaftlicher Betrieb in Mecklenburg ist im Bereich der Produktion von Getreide, Ölfrüchten sowie Tierhaltung tätig. In vielen Bereichen des Betriebs werden IT-gestützte Verfahren verwendet, die aber nicht mehr auf dem aktuellen Stand sind. Daher soll in einem Projekt die Digitalisierung des Betriebs modernisiert werden. Sie arbeiten im Team des Systemhauses AMAG AG mit, das den Betrieb in diesem Projekt unterstützt.

1. Aufgabe (30 Punkte)

Das Systemhaus AMAG AG soll Methoden zur Weiterverarbeitung der Milchproduktionsdaten entwickeln.

List<ProdData>		
date	name	leistung
...	...	...
2026-04-12	Winnie	45.6
2026-04-12	Elsi	39.4
2026-04-12	Rosi	49.8
...	...	...
2026-04-13	Winnie	42.1
2026-04-13	Elsi	40.2
...	...	...



List<KuhData>	
name	avgLeistung
Winnie	44.17
Elsi	38.0
Rosi	47.83
...	...

ProdData	
- date : Date	
- name : String	
- leistung : double	
...	

KuhData	
- name : String	
- avgLeistung : double	
+ KuhData(name : String, avgLeistung : double)	
...	

Die Produktionsdaten liegen als Objekte der Klasse **ProdData** in einer Liste **List<ProdData>** vor. ProdData-Objekte haben als Attribute das Tagesdatum **date**, den **eindeutigen** Kuhnamen **name** und die Milch-Tagesleistung **leistung**.

Benötigt wird eine Liste **List<KuhData>**, die für jede Kuh ein Objekt der Klasse **KuhData** mit den Attributen **name** und der Durchschnittsleistung **avgLeistung** für einen frei wählbaren Zeitraum enthält.

Gehen Sie davon aus, dass für alle Attribute in den jeweiligen Klassen Getter und Setter vorhanden sind.

Für Listen stehen folgende Methoden zur Verfügung.

List<T>	
+ add(t : T) : void	
+ size() : Integer	
+ get(i : Integer) : T	
+ contains(t : T) : Boolean	

- Mit der **add**-Methode wird ein Objekt ans Ende der Liste hinzugefügt.
- Die **size**-Methode liefert die Anzahl der in der Liste vorhandenen Objekte.
- Die **get**-Methode gibt das Objekt an der Position **i** zurück.
- Die **contains**-Methode gibt **true** zurück, wenn die Liste ein namensgleiches Objekt enthält.

Die Klasse **Date** enthält eine Methode **date.between(start, ende)**. Sie liefert

- **true**, wenn **date** im Datumsbereich zwischen **start** und **ende** liegt und
- **false**, wenn **date** **nicht** im Datumsbereich zwischen **start** und **ende** liegt.

Hinweis: **String**-Werte können mit „**==**“ verglichen werden.

Die Aufgabe soll in zwei Schritten a und b erledigt werden.

Korrekturrand

a)

List<ProdData>		
date	name	leistung
...	...	...
2026-04-12	Winnie	45.6
2026-04-12	Elsi	39.4
2026-04-12	Rosi	49.8
...	...	...
2026-04-13	Winnie	42.1
2026-04-13	Elsi	40.2
...	...	...



List<KuhData>	
name	avgLeistung
Winnie	0.0
Elsi	0.0
Rosi	0.0
...	...

Entwickeln Sie eine Methode **erzeugeKuhData**, die für jede Kuh **ein** Objekt der Klasse **KuhData** erzeugt, der Liste **List<KuhData>** hinzufügt und diese Liste abschließend zurückgibt. Die Durchschnittsleistungen **avgLeistung** sollen zunächst mit 0.0 vorinitialisiert werden.

12 Punkte

Hinweis: Die Berechnung der Durchschnittsleistungen erfolgt erst in Teilaufgabe b.

```
List<KuhData> erzeugeKuhData(List<ProdData> prodData) {  
    List<KuhData> kuhData = new List<KuhData>()  

```

```
        return kuhData  
    }
```

Fortsetzung 1. Aufgabe →

Fortsetzung 1. Aufgabe

Korrekturrand

b)

List<KuhData>	
name	avgLeistung
Winnie	0.0
Elsi	0.0
Rosi	0.0
...	...



List<KuhData>	
name	avgLeistung
Winnie	44.17
Elsi	38.0
Rosi	47.83
...	...

Eine Methode **calcAvgLeistung** soll die Durchschnittsleistungen je Kuh berechnen, die mit 0.0 vorinitialisierten Werte damit überschreiben und die geänderte Liste zurückgeben.

Gehen Sie davon aus, dass die Methode **erzeugeKuhData** aus Aufgabe a) vorliegt.

Entwickeln Sie die Methode **calcAvgLeistung**. Der gewünschte Zeitraum wird in start und ende übergeben. 18 Punkte

```
List<KuhData> calcAvgLeistung(List<ProdData> prodData, Date start, Date ende) {  
    List<KuhData> kuhData = erzeugeKuhData(prodData)
```

```
    return kuhData  
}
```

2. Aufgabe (20 Punkte)

Korrekturrand

Im Betrieb soll zur Unterstützung der Aufgabenplanung ein Programm entwickelt werden. Der Entwurf soll objektorientiert erfolgen und Sie arbeiten bei der Entwicklung von Klassendiagramm und Quelltext mit. In der Diskussion wurden folgende Klassen identifiziert:

Es gibt eine allgemeine Klasse **Aufgabe** und Klassen für spezielle Aufgaben, wie:

- **AufgabeFixTermin**
- **AufgabeWetterabhängig**

Es soll bereits jetzt vorgesehen werden, dass später Klassen für weitere Aufgabenarten einfach hinzugefügt werden können.

Attribute der Klasse **Aufgabe**:

- Die Beschreibung der Aufgabe, z. B. „Dünger ausfahren“, wird als Text gespeichert.
- Wie viele Mitarbeiter für die Aufgabe benötigt werden wird als ganze Zahl hinterlegt.

Bei Aufgaben mit festem Termin ist dieses Datum im Attribut **termin** zu speichern.

Bei Aufgaben mit Wettervoraussetzung ist die Art der Wettervoraussetzung zu erfassen (z. B. sonnig, trocken, Regen erwartet, ...).

Der Datentyp **Wetter** soll hierzu verwendet werden.

Methoden der Klassen für Aufgaben:

- **aufgabeBeschreiben**: Gibt einen String mit dem Aufgabentext und der Anzahl benötigter Mitarbeiter zurück.
- **heuteAusführbar**: Prüft, ob zum Aufrufzeitpunkt die Voraussetzungen zum Ausführen erfüllt sind (Wettervoraussetzung erfüllt, oder heute ist gleich dem festen Termin). Später im Programm soll automatisch die zur Klasse passende Methode **heuteAusführbar** aufgerufen werden.

Die Klasse **AufgabenContainer** beinhaltet die Verwaltung der Aufgaben.

Attribut: **List<Aufgabe> aufgaben**

Methoden:

- **aufgabeHinzufügen**: Fügt eine Aufgabe (Objekt der Klasse **Aufgabe**) hinzu.
- **aufgabenHeute**: Gibt eine Liste der heute ausführbaren Aufgaben zurück.

- a) – Zeichnen Sie ein Klassendiagramm mit den Klassen **AufgabenContainer**, **Aufgabe**, **AufgabeFixTermin**, **AufgabeWetterabhängig**.
- Tragen Sie bei jeder Klasse die benötigten Eigenschaften und Methoden mit vollständiger Signatur ein, um die beschriebenen Anforderungen zu erfüllen.
 - Kennzeichnen Sie die Sichtbarkeiten der Eigenschaften und Methoden.
 - Zeichnen Sie die Beziehungen zwischen den Klassen ein. Kardinalitäten sind nicht einzutragen.

12 Punkte

Fortsetzung 2. Aufgabe →

Korrekturrand

4 Punkte

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There is no handwriting or other markings on the paper.

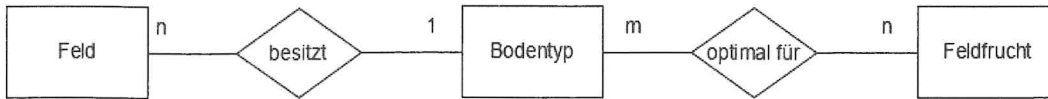
4 Punkte

3. Aufgabe (25 Punkte)

Korrekturrand

- a) Die Daten für die Verwaltung und Bearbeitung der Felder sollen in einer relationalen Datenbank gespeichert werden. Es liegen Ihnen ein ER-Modell und Informationen aus einer Ist-Analyse vor, welche für die Erstellung des relationalen Datenmodells die Grundlage bilden.

ER-Modell:



Informationen aus der Ist-Analyse:

- Beim Anbau von Feldfrüchten auf einzelnen Feldern werden unterschiedliche Bearbeitungsschritte durchgeführt. Für jeden Bearbeitungsschritt sollen das betreffende Feld, die angebaute Feldfrucht sowie der Zeitraum (von/bis) erfasst werden.
- Außerdem kann jedem Bearbeitungsschritt genau eine Tätigkeit wie z.B. Aussaat, Ernte oder ähnliches zugeordnet werden.
- Zu jeder Tätigkeit gibt es eine Beschreibung

Überführen Sie das gegebene ER-Modell in ein relationales Datenbankmodell in der 3. Normalform und erweitern Sie es anhand der Informationen aus der Ist-Analyse. Übernehmen Sie alle in der Ist-Analyse genannten Attribute in das Modell und bestimmen Sie für jede Tabelle einen Primärschlüssel (PK). Bilden Sie alle Beziehungen zwischen den Tabellen unter Angabe der jeweiligen Kardinalitäten ab und tragen Sie die dafür notwendigen Fremdschlüssel (FK) in die Tabellen ein. 20 Punkte

Fortsetzung 3. Aufgabe →

- Berechnen Sie den benötigten Speicherbedarf in TiB, welcher für die Bilder innerhalb eines Jahres (52 Wochen) anfällt. Runden Sie das Ergebnis auf zwei Stellen nach dem Komma. Der Rechenweg ist anzugeben. 5 Punkte

A full-page view of a blank sheet of white graph paper. The grid consists of thin, light gray horizontal and vertical lines forming small squares across the entire page. There are no margins, text, or other markings on the paper.

a) Stellen Sie den Zusammenhang zwischen den SQL-Befehlen mit den grundlegenden Datenbankoperationen (CRUD) her.

- CRUD-Operationen sind grundlegende Funktionen, die in der Softwareentwicklung und Datenbankverwaltung verwendet werden, um mit Daten zu interagieren. CRUD ist ein Akronym, das für die vier grundlegenden Operationen steht.

3 Punkte

ZPA FIA II 8

Dieses Blatt kann an der Perforation aus dem Aufgabensatz herausgetrennt werden!

Die Datenbank der Tiere soll mit SQL abgefragt werden (Aufgaben b und c)

Tabelle: Tierkategorie

TK_ID	TK_Kategorie
1	Kühe
2	Schweine
3	Hühner
...	

Tabelle: Tierbestand

TB_ID	TB_TKID	TB_ChipNr	TB_GebDat	TB_SchlachtDat
1001	2	S03-333-567	03.01.2022	NULL
1002	2	S03-343-566	05.01.2022	NULL
1003	1	K77-899988	12.08.2021	NULL
1004	2	S12-222-322	03.11.2024	NULL
...				

Tabelle: TierZusatzInfo

TZI_ID	TZI_TBID	TZI_ErfasstAm	TZI_Gewicht	TZI_Bemerkung
1	3	12.02.2026	334	ohne Befund
2	1	12.02.2026	655	ohne Befund
3	3	11.03.2026	342	schläfrig
4	3	12.04.2026	344	ohne Befund
...				

Tabelle: Archiv_Tierbestand (wird benötigt für Aufgabe 4 c)

A_TBID	A_Tierkategorie	A_ChipNr	A_GebDat	A_SchlachtDat	A_MaxGewicht
505	Schweine	S02-333-322	01.04.2020	01.04.2023	466
506	Schweine	S02-343-166	05.05.2021	01.04.2023	578
508	Kühe	K55-899777	05.05.2021	05.04.2023	712
...					

b) Erstellen Sie eine SQL-Abfrage mit der Sie eine Auflistung aller Tier-Kategorien mit den nachfolgenden Informationen je Tier-Kategorie über die lebenden Tiere erhalten:

Korrekturrand
8 Punkte

- Anzahl des Tierbestandes
- Gewicht des schwersten Tieres
- Alter des ältesten Tieres
- Durchschnittliches Alter

Ergebnistabelle:

TK_Kategorie	AnzahlTiere	SchwerstesTier	ÄltestesTier	DurchschnittAlter
Hühner	0	NULL	NULL	NULL
Kühe	3	344	4	4
Schweine	3	655	1	2
...				

